

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П.КОРОЛЕВА»

Институт «Информатики и кибернетики»

Специальность «Фотоника и оптоинформатика 6201-120303D»

Отчет по лабораторной работе № 7

Выполнил: студент Султанова А.М.,

группа 6201-120303D

Проверил: преподаватель Борисов Д.С.

Самара 2025

Задание 1

Шаг 1. Обновляем интерфейс TabulatedFunction

```
package functions;

public interface TabulatedFunction extends Function, Cloneable, Iterable<FunctionPoint> { 13 usages 2 implementations
    int getPointsCount(); 4 usages 2 implementations
    double getPointX(int index); 9 usages 2 implementations
    double getPointY(int index); 5 usages 2 implementations
    void setPointX(int index, double x) throws InappropriateFunctionPointException; no usages 2 implementations
    void setPointY(int index, double y); no usages 2 implementations
    FunctionPoint getPoint(int index); 3 usages 2 implementations
    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; no usages 2 implementations
    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException; no usages 2 implementations
    void deletePoint(int index); no usages 2 implementations

    Object clone() throws CloneNotSupportedException; 2 implementations
}
```

Шаг 2. Реализуем итератор в ArrayTabulatedFunction

```
import java.util.Iterator;
import java.util.NoSuchElementException;
```

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private int index = 0; 2 usages

        @Override
        public boolean hasNext() {
            return index < size;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            return new FunctionPoint(points[index++]);
        }

        @Override
        public void remove(){
            throw new UnsupportedOperationException();
        }
    };
}
```

Шаг 3. Реализуем итератор в LinkedListTabulatedFunction

```
import java.util.Iterator;
import java.util.NoSuchElementException;
```

```
@Override
public Iterator<FunctionPoint> iterator() {
    return new Iterator<FunctionPoint>() {
        private FunctionNode node = head.next; 4 usages

        @Override
        public boolean hasNext() {
            return node != head;
        }

        @Override
        public FunctionPoint next() {
            if (!hasNext()) {
                throw new NoSuchElementException("Нет следующего элемента");
            }
            FunctionPoint point = new FunctionPoint(node.point);
            node = node.next;
            return point;
        }

        @Override
        public void remove(){
            throw new UnsupportedOperationException();
        }
    };
}
```

Задание 2

Шаг 1. Создаем интерфейс фабрики

```
package functions;

public interface TabulatedFunctionFactory { no usages new *
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount); no usages new *
    TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values); no usages new *
    TabulatedFunction createTabulatedFunction(FunctionPoint[] points); no usages new *
}
```

Шаг 2. Создаем фабрику для списков (LinkedList)

```
public static class LinkedListTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages
    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new LinkedListTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new LinkedListTabulatedFunction(leftX, rightX, values);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new LinkedListTabulatedFunction(points);
    }
}
```

Шаг 3. Создаем фабрику для массивов (Array)

```
public static class ArrayTabulatedFunctionFactory implements TabulatedFunctionFactory { no usages
    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) {
        return new ArrayTabulatedFunction(leftX, rightX, pointsCount);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) {
        return new ArrayTabulatedFunction(leftX, rightX, values);
    }

    @Override no usages
    public TabulatedFunction createTabulatedFunction(FunctionPoint[] points) {
        return new ArrayTabulatedFunction(points);
    }
}
```

Шаг 4. Обновляем TabulatedFunctions (внедряем фабрику)

```
// Задание 2
private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory(); 4 usages

public static void setTabulatedFunctionFactory(TabulatedFunctionFactory newFactory) { no usages
    factory = newFactory;
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { no usages
    return factory.createTabulatedFunction(leftX, rightX, pointsCount);
}

public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { no usages
    return factory.createTabulatedFunction(leftX, rightX, values);
}

public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { no usages
    return factory.createTabulatedFunction(points);
}
```

В методе `tabulate` заменили строку: `return new ArrayTabulatedFunction(leftX, rightX, yValues);` На: `return factory.createTabulatedFunction(leftX, rightX, yValues);`

Задание 3

Шаг 1. Добавляем методы с рефлексией в `TabulatedFunctions`

```
package functions;

import java.io.*;
import java.lang.reflect.Constructor;

public class TabulatedFunctions { no usages

    private TabulatedFunctions() {} no usages

    // ЗАДАНИЕ 2: ПОЛЕ ФАБРИКИ И МЕТОДЫ РАБОТЫ С НЕЙ

    private static TabulatedFunctionFactory factory = new ArrayTabulatedFunction.ArrayTabulatedFunctionFactory(); 7 usages

    public static void setTabulatedFunctionFactory(TabulatedFunctionFactory newFactory) { no usages
        factory = newFactory;
    }

    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, int pointsCount) { no usages
        return factory.createTabulatedFunction(leftX, rightX, pointsCount);
    }

    public static TabulatedFunction createTabulatedFunction(double leftX, double rightX, double[] values) { no usages
        return factory.createTabulatedFunction(leftX, rightX, values);
    }

    public static TabulatedFunction createTabulatedFunction(FunctionPoint[] points) { no usages
        return factory.createTabulatedFunction(points);
    }

    // ОБЫЧНОЕ ТАБУЛИРОВАНИЕ (Через Фабрику)

    public static TabulatedFunction tabulate(Function function, double leftX, double rightX, int pointsCount) { no usages
        if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
            throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
        }
        if (pointsCount < 2) {
            throw new IllegalArgumentException("Количество точек должно быть не меньше двух");
        }

        double step = (rightX - leftX) / (pointsCount - 1);
        double[] yValues = new double[pointsCount];

        for (int i = 0; i < pointsCount; i++) {
            double x = leftX + i * step;
            yValues[i] = function.getFunctionValue(x);
        }

        return factory.createTabulatedFunction(leftX, rightX, yValues);
    }
}
```

```
// ВВОД/ВЫВОД (Через Фабрику)
```

```
public static void outputTabulatedFunction(TabulatedFunction function, OutputStream out) throws IOException { no usages
    DataOutputStream dos = new DataOutputStream(out);
    dos.writeInt(function.getPointsCount());
    for (int i = 0; i < function.getPointsCount(); i++) {
        dos.writeDouble(function.getPointX(i));
        dos.writeDouble(function.getPointY(i));
    }
    dos.flush();
}

public static TabulatedFunction inputTabulatedFunction(InputStream in) throws IOException { no usages
    DataInputStream dis = new DataInputStream(in);
    int pointsCount = dis.readInt();
    FunctionPoint[] points = new FunctionPoint[pointsCount];

    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
    }

    return factory.createTabulatedFunction(points);
}

public static void writeTabulatedFunction(TabulatedFunction function, Writer out) throws IOException { no usages
    BufferedWriter bw = new BufferedWriter(out);
    bw.write(String.valueOf(function.getPointsCount()));
    bw.write(" ");
    for (int i = 0; i < function.getPointsCount(); i++) {
        bw.write(function.getPointX(i) + " " + function.getPointY(i) + " ");
    }
    bw.newLine();
    bw.flush();
}

public static TabulatedFunction readTabulatedFunction(Reader in) throws IOException { no usages
    StreamTokenizer st = new StreamTokenizer(in);
    st.nextToken();
    int pointsCount = (int) st.nval;

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        st.nextToken();
        double x = st.nval;
        st.nextToken();
        double y = st.nval;
        points[i] = new FunctionPoint(x, y);
    }

    return factory.createTabulatedFunction(points);
}
```

// ЗАДАНИЕ 3: РЕФЛЕКСИЯ (Создание)

```
public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, double leftX, double rightX, int pointsCount) { no usages
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(double.class, double.class, int.class);
        return constructor.newInstance(leftX, rightX, pointsCount);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, double leftX, double rightX, double[] values) { no usages
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(double.class, double.class, double[].class);
        return constructor.newInstance(leftX, rightX, values);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction createTabulatedFunction(Class<? extends TabulatedFunction> functionClass, FunctionPoint[] points) { 3 usages
    try {
        Constructor<? extends TabulatedFunction> constructor = functionClass.getConstructor(FunctionPoint[].class);
        return constructor.newInstance((Object) points);
    } catch (Exception e) {
        throw new IllegalArgumentException(e);
    }
}

public static TabulatedFunction tabulate(Class<? extends TabulatedFunction> functionClass, Function function, double leftX, double rightX, int pointsCount) { no usages
    if (leftX < function.getLeftDomainBorder() || rightX > function.getRightDomainBorder()) {
        throw new IllegalArgumentException("Границы табулирования выходят за область определения функции");
    }

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    double step = (rightX - leftX) / (pointsCount - 1);

    for (int i = 0; i < pointsCount; i++) {
        double x = leftX + i * step;
        points[i] = new FunctionPoint(x, function.getFunctionValue(x));
    }

    return createTabulatedFunction(functionClass, points);
}
```

// ЗАДАНИЕ 3: РЕФЛЕКСИЯ (Ввод/Вывод - добавленные методы)

```
public static TabulatedFunction inputTabulatedFunction(Class<? extends TabulatedFunction> clazz, InputStream in) throws IOException { no usages
    DataInputStream dis = new DataInputStream(in);
    int pointsCount = dis.readInt();
    FunctionPoint[] points = new FunctionPoint[pointsCount];

    for (int i = 0; i < pointsCount; i++) {
        points[i] = new FunctionPoint(dis.readDouble(), dis.readDouble());
    }

    return createTabulatedFunction(clazz, points);
}

public static TabulatedFunction readTabulatedFunction(Class<? extends TabulatedFunction> clazz, Reader in) throws IOException { no usages
    StreamTokenizer st = new StreamTokenizer(in);
    st.nextToken();
    int pointsCount = (int) st.nval;

    FunctionPoint[] points = new FunctionPoint[pointsCount];
    for (int i = 0; i < pointsCount; i++) {
        st.nextToken();
        double x = st.nval;
        st.nextToken();
        double y = st.nval;
        points[i] = new FunctionPoint(x, y);
    }

    return createTabulatedFunction(clazz, points);
}
```

MAIN:

```
1  import functions.*;
2  import functions.basic.Cos;
3  import functions.basic.Sin;
4  import java.util.Iterator;
5
6  public class Main {
7      public static void main(String[] args) {
8
9          System.out.println("Задание 1: Итератор");
10
11         // Создаем уникальные точки
12         FunctionPoint[] myPoints = {
13             new FunctionPoint(x: 1.0, y: 10.0),
14             new FunctionPoint(x: 2.0, y: 20.0),
15             new FunctionPoint(x: 3.0, y: 30.0)
16         };
17
18         // Проверяем LinkedList
19         TabulatedFunction listFunc = new LinkedListTabulatedFunction(myPoints);
20         System.out.println("Перебор списка (LinkedList) через for-each:");
21         for (FunctionPoint p : listFunc) {
22             System.out.println(p);
23         }
24
25         // Проверяем Array и защиту итератора
26         TabulatedFunction arrayFunc = new ArrayTabulatedFunction(myPoints);
27         System.out.println("\nПеребор массива (Array) и тест ошибки удаления:");
28         Iterator<FunctionPoint> iterator = arrayFunc.iterator();
29         while (iterator.hasNext()) {
30             FunctionPoint p = iterator.next();
31             System.out.print(p + " ");
32         }
33         System.out.println();
34
35         try {
36             iterator.remove();
37         } catch (UnsupportedOperationException e) {
38             System.out.println("Успешно: метод remove() выбросил ошибку, как положено.");
39         }
40
41
42         System.out.println("\nЗадание 2: Фабрики");
43         Function sinFunc = new Sin();
44
45         // Устанавливаем фабрику LinkedList
46         System.out.println("Используем LinkedListFactory");
47         TabulatedFunctions.setTabulatedFunctionFactory(new LinkedListTabulatedFunction.LinkedListTabulatedFunctionFactory());
48
49         // Табулируем Синус от 0 до PI/2 с 10 точками
50         TabulatedFunction tf = TabulatedFunctions.tabulate(sinFunc, leftX: 0, rightX: Math.PI / 2, pointsCount: 10);
51         System.out.println("Создан класс: " + tf.getClass().getSimpleName());
52
53         // ПРОВЕРКА ЗНАЧЕНИЙ
54         System.out.println("Проверка значений (Сравнение с Math.sin):");
55         boolean isCorrect = true;
56         // Проверяем каждую точку
57         for (FunctionPoint p : tf) {
```



```

57     for (FunctionPoint p : tf) {
58         double expected = Math.sin(p.getX());
59         // Сравниваем с небольшой погрешностью
60         if (Math.abs(p.getY() - expected) > 1e-9) {
61             System.out.printf("ОШИБКА: x=%.4f, y=%.4f, ожидалось=%.4f%n", p.getX(), p.getY(), expected);
62             isCorrect = false;
63         }
64     }
65     if (isCorrect) {
66         System.out.println("Все точки совпадают с эталонным Синусом!");
67     }
68     // Выведем последнюю точку для наглядности
69     System.out.println("Пример последней точки: " + tf.getPointY(index: tf.getPointsCount() - 1));
70
71
72     System.out.println("\nЗадание 3: Рефлексия ");
73     Function cosFunc = new Cos();
74
75     // 1. Создаем ArrayTabulatedFunction через класс
76     System.out.println("Создание через .class (Array)");
77     TabulatedFunction fReflect = TabulatedFunctions.createTabulatedFunction(
78         ArrayTabulatedFunction.class, leftX: 0, rightX: 3, pointsCount: 3); // 3 точки: 0, 1.5, 3
79     System.out.println("Класс: " + fReflect.getClass().getSimpleName());
80     System.out.println("Точки (должны быть (0,0), (1.5,0), (3,0)): " + fReflect);
81
82     // 2. Создаем LinkedListTabulatedFunction через класс с массивом значений
83     System.out.println("\nСоздание через .class (LinkedList + значения)");
84     double[] values = {11.1, 22.2, 33.3}; // Уникальные значения
85     fReflect = TabulatedFunctions.createTabulatedFunction(
86         LinkedListTabulatedFunction.class, leftX: 0, rightX: 10, values);
87
88     System.out.println("Класс: " + fReflect.getClass().getSimpleName());
89     // Проверяем, записались ли значения
90     if (Math.abs(fReflect.getPointY(index: 1) - 22.2) < 1e-9) {
91         System.out.println("Значение посередине корректно: " + fReflect.getPointY(index: 1));
92     } else {
93         System.out.println("ОШИБКА: значение не совпало!");
94     }
95
96     // 3. Табулируем Косинус через рефлексию
97     System.out.println("\nТабулирование Косинуса через рефлексию (LinkedList)");
98     TabulatedFunction tabCos = TabulatedFunctions.tabulate(
99         LinkedListTabulatedFunction.class, cosFunc, leftX: 0, Math.PI, pointsCount: 11);
100     System.out.println("Создан класс: " + tabCos.getClass().getSimpleName());
101
102     // ПРОВЕРКА ЗНАЧЕНИЙ
103     System.out.println("Проверка значений (Сравнение с Math.cos):");
104     isCorrect = true;
105     for (FunctionPoint p : tabCos) {
106         double expected = Math.cos(p.getX());

```

```

106         double expected = Math.cos(p.getX());
107         if (Math.abs(p.getY() - expected) > 1e-9) {
108             System.out.printf("ОШИБКА: x=%.4f, y=%.4f, ожидалось=%.4f%n", p.getX(), p.getY(), expected);
109             isCorrect = false;
110         }
111     }
112     if (isCorrect) {
113         System.out.println("Все точки совпадают с эталонным Косинусом!");
114     }
115     // Выведем значение в точке 0 (должно быть 1.0)
116     System.out.println("Значение в точке 0: " + tabCos.getPointY(index: 0));
117 }
118 }

```

```
"C:\Program Files\Java\jdk-24\bin\java.exe" "-javaagent:D:\java\IntelliJ IDEA Community Editi
```

Задание 1: Итератор

Перебор списка (LinkedList) через for-each:

(1.0; 10.0)

(2.0; 20.0)

(3.0; 30.0)

Перебор массива (Array) и тест ошибки удаления:

(1.0; 10.0) (2.0; 20.0) (3.0; 30.0)

Успешно: метод remove() выбросил ошибку, как положено.

Задание 2: Фабрики

Используем LinkedListFactory

Создан класс: LinkedListTabulatedFunction

Проверка значений (Сравнение с Math.sin):

Все точки совпадают с эталонным Синусом!

Пример последней точки: 1.0

Задание 3: Рефлексия

Создание через .class (Array)

Класс: ArrayTabulatedFunction

Точки (должны быть (0,0), (1.5,0), (3,0)): {(0.0; 0.0), (1.5; 0.0), (3.0; 0.0)}

Создание через .class (LinkedList + значения)

Класс: LinkedListTabulatedFunction

Значение посередине корректно: 22.2

Табулирование Косинуса через рефлексию (LinkedList)

Создан класс: LinkedListTabulatedFunction

Проверка значений (Сравнение с Math.cos):

Все точки совпадают с эталонным Косинусом!

Значение в точке 0: 1.0

Process finished with exit code 0